

Развёртывание цифрового двойника лаборатории робототехники

ROS2 Humble + Gazebo Classic + WSL2

Цель работы: Изучение принципов построения симуляционных сред на базе ROS2 Humble и Gazebo Classic. Освоение структуры ROS2-пакета типа `ament_python`, создание трёхмерной модели помещения из Blender-меша в формате COLLADA (.dae), настройка физики и освещения сцены, запуск и верификация симуляционного окружения цифрового двойника лаборатории робототехники.

Ход работы

1.1. Концепция цифрового двойника

Цифровой двойник (Digital Twin) — виртуальная копия физического объекта или системы, функционирующая в реальном времени и обменивающаяся данными с реальным аналогом. В контексте лаборатории робототехники цифровой двойник решает три задачи:

1. **Симуляция среды** — точное воспроизведение геометрии и физических свойств помещения: стен, пола, мебели, препятствий.
2. **Тестирование алгоритмов** — проверка навигации, SLAM и управления без риска повреждения реального оборудования.
3. **Интеграционная платформа** — единый ROS2-интерфейс для реального и виртуального робота. Один и тот же код запускается в симуляции и на железе без изменений.

1.2. Архитектура ROS2-пакета (`ament_python`)

ROS2 использует систему сборки `ament` поверх `colcon`. Пакет типа `ament_python` описывается тремя ключевыми файлами.

1. **package.xml** — декларирует имя пакета, версию и зависимости от других ROS2-пакетов. Используется системой управления зависимостями `rosdep`.
2. **setup.py** — стандартный Python `setuptools`, расширенный полем `data_files`. Это поле критически важно: именно оно указывает `colcon`, какие ресурсы (`world`-файлы, меши, конфигурации) скопировать в директорию установки при сборке. Без корректного `data_files` Gazebo не найдёт `world`-файл и модели после `colcon build`.
3. **setup.cfg** — задаёт путь установки исполняемых скриптов (entry points) в `lib/<package_name>/`.

Стандартная структура пакета `lab_digital_twin`:

`src/lab_digital_twin/`

```

└─ lab_digital_twin/      # Python-модуль пакета
|   └─ scripts/brain_node.py # исполняемая нода
└─ launch/               # launch-файлы
└─ worlds/               # .world файлы Gazebo
└─ models/lab_interior/  # SDF-модель помещения
|   └─ meshes/lab_mesh.dae # ← меш из Blender
└─ config/               # YAML параметры нод
└─ resource/lab_digital_twin # обязательныйament-маркер
└─ package.xml
└─ setup.py
└─ setup.cfg

```

1.3. Формат SDF и подключение COLLADA-меша

SDF (Simulation Description Format) — основной формат описания моделей в Gazebo. Каждая модель содержит ссылки (link) с двумя обязательными компонентами.

<visual> — геометрия, используемая только для рендеринга. Не влияет на физику. Здесь указывается путь к .dae файлу через URI вида `model://lab_interior/meshes/lab_mesh.dae`.

<collision> — геометрия для расчёта физических столкновений. Можно использовать тот же меш, но для производительности рекомендуется упрощённая геометрия — набор `<box>`-примитивов под каждую стену.

Чтобы Gazebo нашёл модель по URI `model://`, необходимо зарегистрировать путь к папке `models/` через переменную окружения:

bash

export

```
GAZEBO_MODEL_PATH=~/.ros2_ws/install/lab_digital_twin/share/lab_digital_twin/models:$GAZEBO_MODEL_PATH
```

Эта переменная автоматически выставляется в `simulation.launch.py` через `os.environ` при запуске, поэтому вручную прописывать её нужно только при отладке.

1.4. Физика и освещение в Gazebo

Файл `.world` описывает полное окружение симуляции. Ключевые разделы:

<physics> — задаёт физический движок (ODE — Open Dynamics Engine) и параметры интегрирования. Параметр `max_step_size = 0.001` (1 мс) обеспечивает баланс между точностью и скоростью. При `real_time_factor = 1.0` симуляция идёт в реальном времени.

<light> — источники освещения трёх типов: `directional` (направленный, без ослабления с расстоянием), `point` (точечный, ослабевает с расстоянием), `spot` (прожектор с конусом). В лабораторном мире используются: `ambient` (рассеянный фон), `sun` (основной направленный) и `ceiling_light` (потолочный точечный).

<scene> — настройки рендеринга: цвет фона, включение теней, отображение координатной сетки.

1.5. Launch-файлы в ROS2

Launch-файл в ROS2 — это Python-скрипт, возвращающий объект `LaunchDescription`.
Ключевые абстракции:

DeclareLaunchArgument — объявляет параметр, который можно переопределить при запуске: `ros2 launch pkg file.launch.py gui:=false`.

IncludeLaunchDescription — включает другой launch-файл, в данном случае стандартный `gazebo.launch.py` из пакета `gazebo_ros`.

Node — запускает ROS2-ноду с заданным пакетом, исполняемым файлом и параметрами.

LogInfo — выводит сообщение в консоль в момент старта системы.

2.1. Подготовка окружения WSL2 и ROS2

Все операции выполняются внутри WSL2 (Ubuntu 22.04).

Шаг 1. Установить ROS2 Humble и необходимые пакеты:

```
bash
```

```
sudo apt update
```

```
sudo apt install -y ros-humble-desktop ros-humble-gazebo-ros-pkgs \
python3-colcon-common-extensions
```

Шаг 2. Добавить автозагрузку окружения:

```
bash
```

```
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
```

```
source ~/.bashrc
```

Шаг 3. Проверить установку:

bash

ros2 --version

gazebo --version

2.2. Создание рабочего пространства и сборка пакета

Шаг 1. Создать workspace и скопировать пакет:

bash

mkdir -p ~/ros2_ws/src

cp -r /mnt/f/lab_digital_twin ~/ros2_ws/src/

Шаг 2. Создать заглушку меша до получения реального .dae из Blender:

bash

touch ~/ros2_ws/src/lab_digital_twin/models/lab_interior/meshes/lab_mesh.dae

Шаг 3. Собрать пакет:

bash

cd ~/ros2_ws

source /opt/ros/humble/setup.bash

colcon build --packages-select lab_digital_twin --symlink-install

source install/setup.bash

Шаг 4. Проверить что ресурсы скопированы корректно:

bash

ls install/lab_digital_twin/share/lab_digital_twin/

Ожидаем: launch/ worlds/ models/ config/

2.3. Подготовка 3D-меша в Blender

Шаг 1. Открыть сцену лаборатории в Blender.

Шаг 2. Перейти: File → Export → Collada (.dae).

Шаг 3. Настройки экспорта: установить флаги Apply Modifiers и Include UV Textures, Scale = 1.0 (сцена в метрах) или Scale = 0.01 если сцена создана в сантиметрах.

Шаг 4. Сохранить файл как lab_mesh.dae в директорию:

bash

~/ros2_ws/src/lab_digital_twin/models/lab_interior/meshes/

Шаг 5. Пересобрать пакет (повторить шаг 2.3).

2.4. Запуск симуляции

Базовый запуск с графическим интерфейсом:

bash

```
ros2 launch lab_digital_twin simulation.launch.py
```

Запуск без GUI (headless-режим, для слабого железа или удалённой работы):

bash

```
ros2 launch lab_digital_twin simulation.launch.py gui:=false
```

Запуск с подробным выводом для диагностики ошибок загрузки меша:

bash

```
ros2 launch lab_digital_twin simulation.launch.py verbose:=true
```

При успешном запуске открывается окно Gazebo со сценой лаборатории. В консоли появляются сообщения brain_node об инициализации и подписке на топики.

2.5. Верификация развёртывания

Открыть второй терминал WSL2, применить окружение и выполнить проверочные команды:

bash

```
source ~/ros2_ws/install/setup.bash
```

Команда	Ожидаемый результат
ros2 --version	Вывод версии ros2cli
gazebo --version	Gazebo multi-robot simulator, version 11.x
colcon build	Сборка без ошибок, папка install/ создана
ros2 pkg list grep lab	Пакет lab_digital_twin присутствует

Вывод : создали начальное программное окружения для инициализации утилит нашего будущего робота